Documentação Técnica: Codificador Manchester Diferencial

1. Visão geral

O módulo `manchester_dif` implementa um codificador de linha Manchester Diferencial. Cada intervalo de bit é dividido em duas metades e **sempre há uma inversão do nível no meio do bit**, usada apenas para sincronismo. O valor do bit é representado pela presença ou ausência de inversão no **início** do intervalo:

Bit	Início do intervalo	Meio do intervalo
0	ocorre inversão do nível	ocorre inversão do nível
1	mantém o nível anterior	ocorre inversão do nível

Como a regra depende do nível anterior, o circuito memoriza o último nível transmitido. **O nível da saída após o reset é baixo.**

O circuito recebe uma sequência de 16 bits pelos switches da placa Basys 3 e a transmite de forma cíclica, do bit mais significativo (SW15) ao menos significativo (SW0). O sinal codificado é apresentado no LED que fica acima do switch em transmissão e na porta `data_out`, que pode ser levada ao conector PMOD.

2. Diagrama de blocos

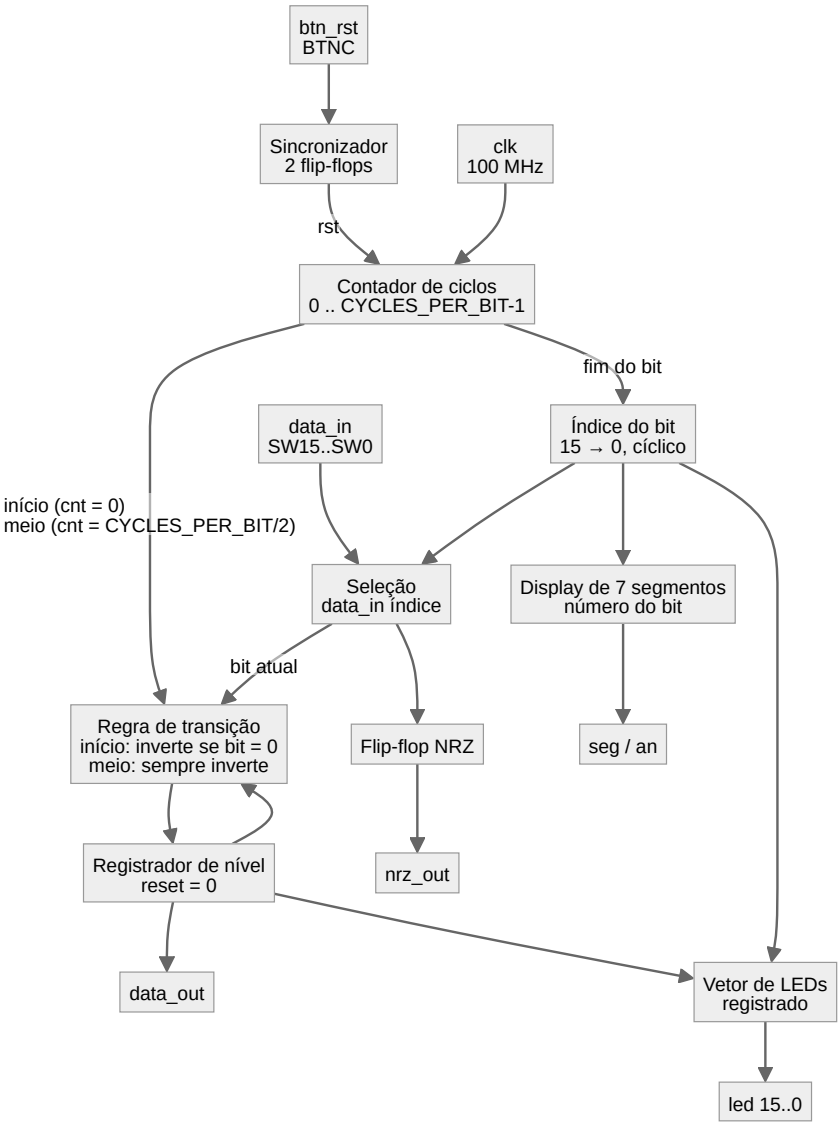


Figura: diagrama de blocos do circuito. A fonte editável do diagrama está em `docs/midia/diagrama_blocos.mmd` (formato Mermaid).

3. Interface

Porta	Direção	Tipo	Função
`clk`	entrada	`std_logic`	Clock de 100 MHz da placa
`btn_rst`	entrada	`std_logic`	Reset (botão BTNC); enquanto pressionado, a transmissão fica parada e a saída em nível baixo
`data_in`	entrada	`std_logic_vector(15 downto 0)`	Sequência de bits (switches SW15..SW0, para cima = 1)
`data_out`	saída	`std_logic`	Sinal codificado em Manchester Diferencial
`nrz_out`	saída	`std_logic`	Bit em transmissão, no formato NRZ (dado original)
`led`	saída	`std_logic_vector(15 downto 0)`	LEDs LD15..LD0; só o LED do bit em transmissão recebe o sinal codificado
`seg`	saída	`std_logic_vector(6 downto 0)`	Segmentos g..a do display de 7 segmentos (ativos em 0)
`an`	saída	`std_logic_vector(3 downto 0)`	Anodos AN3..AN0 do display (ativos em 0)

A interface é idêntica à do módulo `manchester`, o que permite reaproveitar os tops das saídas PMOD e VGA.

4. Parâmetros (generics)

Generic	Tipo	Valor padrão	Função
`CYCLES_PER_BIT`	`positive`	`100_000_000`	Duração de cada bit em ciclos de clock. **Deve ser par**, para que as duas metades do bit tenham a mesma duração; um `assert` interrompe a síntese e a simulação caso contrário

Valores usados no projeto:

Uso	`CYCLES_PER_BIT`	Duração do bit (clock de 100 MHz)
Placa (LEDs)	`100_000_000`	1 s (0,5 s por metade)
Saída PMOD (osciloscópio)	`100_000`	1 ms (taxa de 1 kbit/s)
Testbench	`10`	100 ns (clock simulado de 10 ns)

5. Funcionamento interno

Sincronizador, contador de ciclos e índice do bit. Funcionam como no projeto Manchester: o BTNC passa por dois flip-flops antes de ser usado como reset síncrono; o contador vai de 0 a `CYCLES_PER_BIT - 1`; o índice começa em 15, decrementa a cada fim de bit e volta a 15 após o bit 0.

Registrador de nível. Guarda o último nível transmitido e é, ao mesmo tempo, o flip-flop que alimenta `data_out`. A cada ciclo de clock:

- **contador = 0 (início do bit):** lê `data_in(índice)`. Se o bit for 0, o nível é invertido; se for 1, o nível é mantido. O valor lido também é guardado no flip-flop de `nrz_out`;
- **contador = `CYCLES_PER_BIT/2` (meio do bit):** o nível é sempre invertido;
- **demais ciclos:** o nível é mantido.

Como o switch só é lido no início do bit, alterá-lo no meio do intervalo não afeta o bit em transmissão.

Exemplo. Para a sequência 1 0 1 1 a partir do reset (nível baixo):

Bit	Nível anterior	Início do bit	1ª metade	2ª metade
1	0	mantém	0	1
0	1	inverte	0	1
1	1	mantém	1	0
1	0	mantém	0	1

Saídas registradas. O sinal codificado, o dado original e o vetor de LEDs saem de flip-flops, evitando *glitches* no osciloscópio. O vetor de LEDs é zerado a cada ciclo e apenas a posição do índice recebe o nível atual.

Reset. Enquanto o reset está ativo, o contador vai para 0, o índice para 15 e o registrador de nível para **nível baixo**, assim como todas as saídas. Ao soltar o botão, a transmissão começa em SW15 aplicando a regra de transição a partir do nível baixo. A latência entre soltar o botão e o primeiro bit na saída é de 3 ciclos de clock.

Display de 7 segmentos. O submódulo `display_7seg` recebe o índice do bit e o mostra em decimal nos dois displays da direita (AN1 = dezena, AN0 = unidade), apagando a dezena para números menores que 10. Como os quatro displays compartilham os segmentos, eles são acesos alternadamente: o bit mais significativo de um contador de 17 bits escolhe entre unidade e dezena, de modo que cada display é atualizado a cerca de 380 Hz, sem cintilação perceptível. Durante o reset, o display fica apagado.

Variações para as saídas PMOD e VGA. O mesmo módulo é instanciado, sem alterações, pelos tops das pastas `saidas/`: na saída PMOD, com `CYCLES_PER_BIT = 100_000` (1 ms por bit) e as saídas `data_out` e `nrz_out` ligadas aos pinos JA1 e JA2; na saída VGA, com 1 s por bit e `data_out` ligado ao controlador VGA, que pinta a tela de verde (nível alto) ou azul (nível baixo).

6. Mapeamento físico na placa

Sinal	Componente	Pino
`clk`	Oscilador de 100 MHz	W5
`btn_rst`	Botão central (BTNC)	U18
`data_in(0..15)`	SW0..SW15	V17, V16, W16, W17, W15, V15, W14, W13, V2, T3, T2, R3, W2, U1, T1, R2
`led(0..15)`	LD0..LD15	U16, E19, U19, V19, W18, U15, U14, V14, V13, V3, W3, U3, P3, N3, P1, L1
`data_out`	PMOD JA1	J1
`nrz_out`	PMOD JA2	L2
`seg(0..6)`	Segmentos CA..CG	W7, W6, U8, V8, U5, V5, U7
`an(0..3)`	Anodos AN0..AN3	U2, U4, V4, W4

Os pinos seguem o arquivo `Basys-3-Master.xdc` da Digilent.

Os tops das saídas usam, além desses, os seguintes pinos (arquivos em `saidas/*/constraints/`):

Saída	Sinal	Pinos
PMOD	`ja(0)` (sinal codificado) / `ja(1)` (NRZ)	J1 / L2 (JA1 / JA2)
VGA	`vgaRed(0..3)`	G19, H19, J19, N19
VGA	`vgaGreen(0..3)`	J17, H17, G17, D17
VGA	`vgaBlue(0..3)`	N18, L18, K18, J18
VGA	`Hsync` / `Vsync`	P19 / R19

7. Dependências e ambiente

Item	Valor
Ferramenta	AMD Vivado 2025.2
Simulador	Vivado Simulator
Placa	Digilent Basys 3 (Artix-7 `xc7a35tcpg236-1`)
Linguagem	VHDL-93
Bibliotecas	`ieee.std_logic_1164` e `ieee.numeric_std` (no display)